

Local vs Global Variables in Python

Variables in Python have a scope.

Scope defines where a variable can be accessed in the code.

The two most common scopes are local and global.

Local Variables

Local variables are created inside a function.

They can be used only within that function.

```
def show_value():
```

```
    x = 10
```

```
    print(x)
```

```
show_value()
```

Here, x is a local variable.

If you try to access it outside the function, it will cause an error.

```
print(x)
```

Global Variables

Global variables are created outside all functions. They can be accessed from anywhere in the program.

```
x = 20
```

```
def show_value():
```

```
    print(x)
```

```
show_value()
```

Here, x is a global variable.

Local vs Global Example

```
x = 10
```

```
def test():
```

```
    x = 5
```

```
    print(x)
```

```
test()
```

```
print(x)
```

Output:

- Inside function: 5
- Outside function: 10

The local variable does not affect the global variable.

The global Keyword

The global keyword is used to modify a global variable inside a function.

```
x = 10
```

```
def update():
```

```
    global x
```

```
    x = 20
```

```
update()
```

```
print(x)
```

Without the global keyword, Python treats x as a local variable.

Why global Should Be Used Carefully

Using global variables can make code harder to understand and debug.

It is generally better to:

- Pass values as parameters
- Return values from functions

Global Variables and Function Parameters

A better approach than using global is passing data explicitly.

```
def update(x):
```

```
    return x + 10
```

```
value = 5
```

```
value = update(value)
```

Variable Scope Rules

- Local variables exist only inside functions
- Global variables exist throughout the program
- Local variables override global variables with the same name
- The global keyword allows modifying global variables